

Manual de Usuario y Operación

Plataforma de Procesamiento Distribuido Multimedia (Python + FastAPI + Tailscale)

1. Arquitectura general

La plataforma procesa archivos multimedia (MP3, MP4 y WAV) repartiendo el trabajo entre un **Servidor Central** y varias computadoras **Worker** conectadas por la red privada de **Tailscale**. Cada worker pide una tarea, procesa el archivo con **FFmpeg** y reporta el resultado; el servidor reparte las tareas, reintenta las que fallan y recupera las que un worker abandonó. El estado se sigue en tiempo real desde el **Dashboard**.

Componente	Archivo	Función
Servidor Central	<code>server.py</code>	API REST (FastAPI) con la cola de tareas en SQLite (<code>tasks.db</code> , se crea junto a <code>server.py</code>). Asigna tareas sin duplicados, registra resultados y tiempos, y aplica la tolerancia a fallos.
Worker	<code>worker.py</code>	Corre en cada computadora. Procesa con FFmpeg: mp4 a video H.264 + AAC (<code>transcode_video</code>); mp3 y wav a MP3 (<code>convert_audio</code>). Guarda los resultados en <code>resultados_worker/</code> .
Sembrador	<code>seed_tasks.py</code>	Escanea la carpeta del dataset y registra cada archivo como tarea. Con <code>--reset</code> vacía la cola antes de registrar.
Dashboard	<code>dashboard/app.py</code>	Monitor web (Streamlit): estado de la cola, actividad por worker y explorador de tareas.
Red privada	Tailscale	Conecta servidor y workers con IPs privadas <code>100.x.y.z</code> , sin abrir puertos en el router.

2. Requisitos

Máquina	Necesita
Servidor Central	Python 3.13 o superior, el repositorio y sus dependencias (<code>pip install -r requirements.txt</code>), Tailscale con sesión iniciada. Una copia del dataset para sembrar la cola.
Cada Worker	Python, el repositorio y sus dependencias, FFmpeg (comandos <code>ffmpeg</code> y <code>ffprobe</code> en el PATH), una copia local del dataset y Tailscale con la misma cuenta.
Dashboard	Las dependencias de <code>requirements-dashboard.txt</code> (Streamlit y pandas).

Instalar FFmpeg:

- Windows: `winget install --id Gyan.FFmpeg -e`. Si winget no está disponible o falla, el script `documentacion/UNIRSE_COMO_WORKER_WINDOWS.ps1` lo descarga e instala automáticamente.
- Linux (Debian/Ubuntu): `sudo apt install -y ffmpeg`.

- Comprobar: `ffmpeg -version`.

3. Dataset

El dataset está en `C:\dataset` (Windows) organizado por formato: `mp3s\`, `mp4\` y `wav\` (307 archivos). El servidor registra cada archivo por su ruta relativa, por ejemplo `mp4/mp4s/7332159553984843013.mp4`.

- Cada worker procesa archivos de **su propia copia** del dataset (indicada con `--dataset-dir`).
- La estructura de carpetas puede ser distinta en cada máquina: si la ruta no coincide, el worker busca el archivo por su nombre.
- Si un worker no tiene el archivo, la tarea se reporta como fallida y vuelve a la cola para que la tome otro worker.

4. Red privada con Tailscale

- Instalar Tailscale en el servidor y en cada worker (tailscale.com/download; en Linux: `curl -fsSL https://tailscale.com/install.sh | sh` y luego `sudo tailscale up`).
- Iniciar sesión con la **misma cuenta** en todas las máquinas.
- Obtener la IP del servidor con `tailscale ip -4` (por ejemplo `100.118.70.69`).
- Desde cada worker, verificar el enlace con `tailscale ping <IP-del-servidor>` (debe responder *pong*) y `tailscale status` (debe listar el servidor).

5. Operación paso a paso

Orden recomendado: 1) iniciar el servidor, 2) iniciar todos los workers, 3) sembrar la cola, 4) abrir el dashboard. Si se siembra antes de que los workers se conecten, el primero que llegue toma casi todas las tareas.

Copiar comandos: cada comando es una sola línea. Si al pegarlo queda partido, copiarlo desde `documentacion/REINICIAR_SERVIDOR.txt` o `documentacion/INICIAR_WORKERS.txt`.

Paso 1: Iniciar el Servidor Central

En la carpeta del repositorio, con el entorno virtual activo. Dejar la ventana abierta:

```
python -m uvicorn server:app --host 0.0.0.0 --port 8000
```

Verificar (en Windows: `Invoke-RestMethod http://127.0.0.1:8000/tasks/status`):

```
curl http://127.0.0.1:8000/tasks/status
```

Paso 2: Iniciar los Workers

Windows (recomendado): abrir PowerShell, pegar completo el contenido de `documentacion/UNIRSE_COMO_WORKER_WINDOWS.ps1` (con la IP del servidor en la línea `$Servidor`). El script clona o actualiza el repositorio, crea el entorno, instala dependencias y FFmpeg, verifica el dataset y la conexión, e inicia el worker con un nombre derivado de la computadora.

Comando manual (una sola línea; cada worker con un `--worker-id` distinto):

```
python worker.py --server http://100.118.70.69:8000 --worker-id laptop-nahum-01 --dataset-dir C:\dataset
```

Linux:

```
python3 worker.py --server http://100.118.70.69:8000 --worker-id laptop-kenny-01 --dataset-dir dataset/real
```

Worker local en el propio servidor (opcional):

```
python worker.py --server http://127.0.0.1:8000 --worker-id local-server-worker --dataset-dir C:\dataset
```

Al arrancar, el worker muestra **Modo: REAL (FFmpeg)** y cuántos archivos encontró en el dataset. Si falta FFmpeg o la carpeta del dataset, se detiene con un mensaje que indica qué instalar.

Paso 3: Sembrar la cola

En una segunda ventana del servidor, con los workers ya corriendo:

```
python seed_tasks.py --server http://127.0.0.1:8000 --dataset-dir C:\dataset
```

Debe terminar con *"307 nuevos en cola (pending), 0 ya estaban registrados, 0 errores"*.

Paso 4: Abrir el Dashboard

La primera vez se instalan las dependencias; luego se inicia con:

```
pip install -r requirements-dashboard.txt
streamlit run dashboard/app.py
```

Se abre en `http://localhost:8501`. En la barra lateral se indica la URL del servidor (por defecto `http://localhost:8000`). El dashboard detecta el tipo de servidor y muestra:

Página	Contenido
Resumen	Totales de la cola (pendientes, en proceso, completadas, fallidas), avance, workers activos, gráfico de progreso y tareas en proceso.
Workers	Por cada worker: estado (Activo si procesa o tuvo actividad en los últimos 60 s), tareas completadas y fallidas, tiempo total y promedio, última actividad.
Tareas	Explorador de todas las tareas con filtros por estado, worker, tipo de archivo y búsqueda por nombre; muestra tiempos, reintentos y errores.

Paso 5: Resultados

Cada worker guarda los archivos procesados en su máquina, en `resultados_worker/tarea_<id>/`. El avance global se consulta en el dashboard o con `/tasks/status`.

6. Volver a procesar: reiniciar la cola

Cuando todas las tareas están completadas, los workers muestran *"Sin tareas pendientes"*. Volver a sembrar `sin --reset` no las pone de nuevo en cola: los archivos ya registrados se ignoran y el sembrador lo advierte (*"0 nuevos en cola"*).

Método A (recomendado), sin detener el servidor:

```
python seed_tasks.py --server http://127.0.0.1:8000 --dataset-dir C:\dataset --reset
```

Debe indicar *"Cola reiniciada: N tareas anteriores eliminadas"* y luego *"307 nuevos en cola (pending)"*. Los workers en ejecución toman las tareas automáticamente.

Método B, reinicio completo: detener el servidor (Ctrl + C), borrar `tasks.db` (en la carpeta de `server.py`), iniciar el servidor de nuevo y sembrar `sin --reset`. Las instrucciones detalladas para copiar y pegar están en `documentacion/REINICIAR_SERVIDOR.txt` y `documentacion/INICIAR_WORKERS.txt`.

7. Opciones del Worker

Opción	Descripción
<code>--server</code>	URL del Servidor Central, por ejemplo <code>http://100.118.70.69:8000</code> .
<code>--worker-id</code>	Nombre único del worker (aparece en el dashboard).
<code>--dataset-dir</code>	Carpeta local con la copia del dataset.
<code>--output-dir</code>	Carpeta de resultados (por defecto <code>resultados_worker</code>).
<code>--operation</code>	auto (por defecto: mp4 a <code>transcode_video</code> , mp3/wav a <code>convert_audio</code>) o una operación fija: <code>transcode_video</code> , <code>extract_audio</code> , <code>generate_thumbnail</code> , <code>convert_audio</code> , <code>extract_metadata</code> .
<code>--simulate</code>	No usa FFmpeg: solo espera unos segundos (para probar la conexión).
<code>--simulate-failures</code>	Hace fallar ~15 % de las tareas para probar la tolerancia a fallos.
<code>--poll-interval</code>	Segundos entre consultas cuando la cola está vacía (por defecto 3).

8. Endpoints de la API REST (Swagger en /docs)

Método	Endpoint	Descripción
GET	/tasks/next?worker_id=...	Asigna la siguiente tarea pendiente al worker (nunca la misma a dos workers).
POST	/tasks/{id}/report	Recibe el resultado (completed o failed), el tiempo de ejecución y el error.
POST	/tasks/register	Registra un archivo; responde created = true si es nuevo.
POST	/tasks/scan_dataset	Escanea una carpeta de la máquina del servidor y registra sus archivos.
POST	/tasks/reset	Vacía la cola; opcionalmente re-escanea una carpeta (dataset_path).
GET	/tasks/status	Totales: pendientes, en proceso, completadas, fallidas.
GET	/tasks?status=&limit=	Lista de tareas, de la más reciente a la más antigua.
GET	/tasks/workers	Actividad por worker: en proceso, completadas, fallidas, tiempos y última actividad.

9. Tolerancia a fallos

- **Worker caído o desconectado:** si una tarea queda en proceso más de 5 minutos (300 s), vuelve a pending con un reintento más y la toma otro worker. La revisión ocurre cada vez que algún worker pide una tarea.
- **Límite de reintentos:** una tarea que falla 3 veces se marca como failed y se guarda el error; la cola sigue avanzando.
- **Asignación sin duplicados:** aunque varios workers pidan tareas al mismo tiempo, cada tarea se asigna a uno solo.
- **Errores de procesamiento:** archivos corruptos, formatos sin el stream necesario o faltantes se reportan como fallidos con el tipo de error (por ejemplo `CorruptInputError`).
- **Reconexión:** si el servidor se reinicia, los workers reintentan cada 5 segundos y continúan solos.

10. Solución de problemas

Síntoma	Causa y solución
"Sin tareas pendientes" todo el tiempo	La cola está vacía o ya se procesó. Revisar <code>/tasks/status</code> ; si <i>pending</i> es 0, reiniciar la cola con <code>seed_tasks.py ... --reset</code> .
"No se pudo conectar con el servidor central"	Tailscale apagado en alguna máquina o el servidor detenido. Verificar con <code>tailscale ping</code> .
El worker se detiene: "No se encontró ffmpeg"	Instalar FFmpeg (sección 2) y abrir una terminal nueva, o usar <code>--simulate</code> .
Tareas fallidas: "Archivo no encontrado en --dataset-dir"	Ese worker no tiene una copia del archivo. Copiar el dataset completo o corregir <code>--dataset-dir</code> .
"error: argument --dataset-dir: expected one argument"	El comando se partió en dos líneas al copiarlo. Pegarlo en una sola línea.
<code>git pull</code> falla por <code>tasks.db</code>	La base ya no está en git. Renombrar el archivo local (por ejemplo a <code>tasks_viejo.db</code>) y repetir.
El dashboard muestra "Conexión"	No hay servidor en la URL indicada. Iniciar el servidor (paso 1) o corregir la URL en la barra lateral.